

EXP NO 2	PaaS IN PUBLIC CLOUD USING RENDER
Date: 08.07.2026	

Aim

To deploy a web application on a Platform-as-a-Service (PaaS) cloud environment using Render, and to understand how PaaS enables application deployment and scaling without the need to manage the underlying servers or infrastructure.

Introduction to PaaS and Render

Platform as a Service (PaaS) is a cloud computing model that provides a ready-to-use platform for developing, deploying, and managing applications, without requiring the user to manage the underlying operating system, servers, or networking infrastructure. The cloud provider handles provisioning, scaling, patching, and runtime management, allowing developers to focus purely on application code.

Render is a modern PaaS platform that allows developers to build, deploy, and host web applications, APIs, static sites, and background services directly from a GitHub or GitLab repository. Render automatically builds the application, provisions the required infrastructure, and exposes the deployed application through a public URL.

Key benefits of using a PaaS platform such as Render include:

- No manual server setup, patching, or operating-system management
- Automatic builds and deployments triggered from source-code repository changes
- Built-in HTTPS, load balancing, and auto-scaling
- Faster development-to-deployment cycle compared to IaaS

Software / Environment Used

- Cloud Platform: Render (PaaS)
- Source Control: GitHub repository
- Application: Sample Node.js / static web application
- Interface: Render Web Dashboard

Procedure

Step 1: Create a Render Account

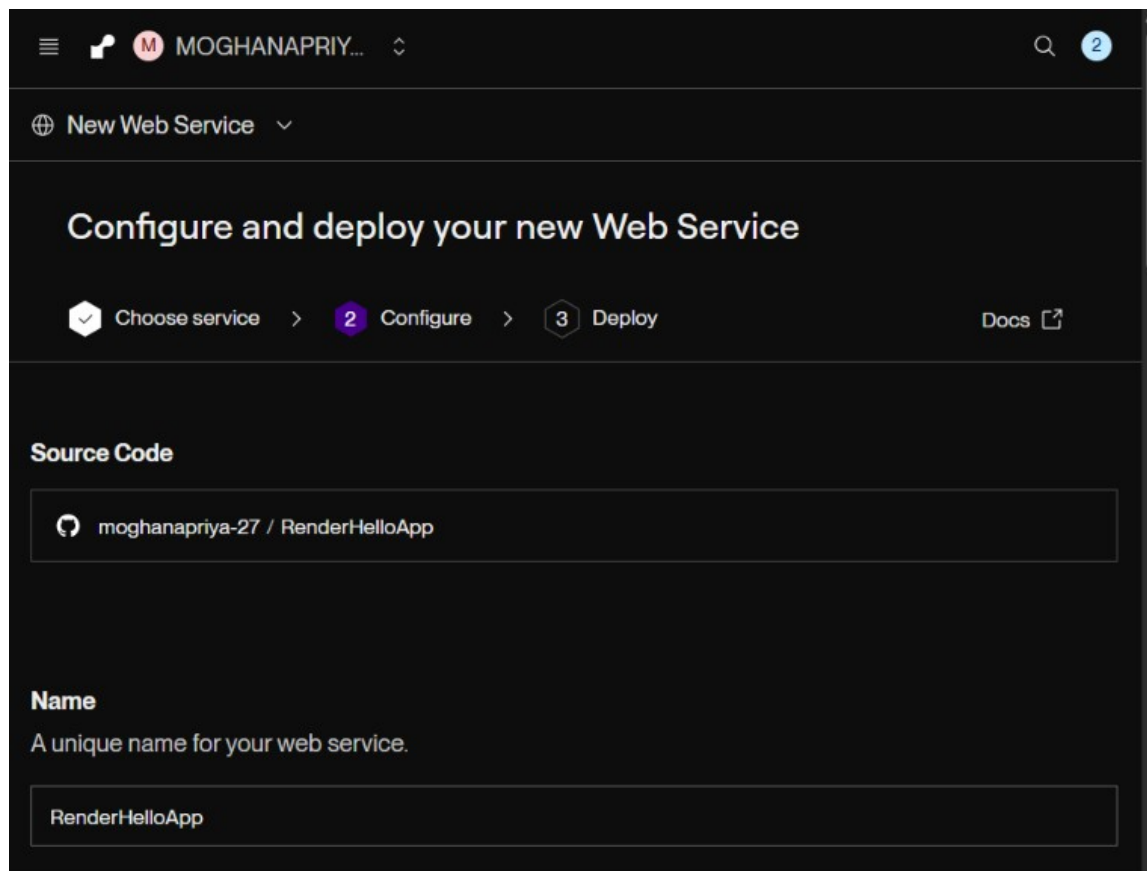
A Render account was created at render.com using a valid email address, and the account was verified and signed in to the Render dashboard.

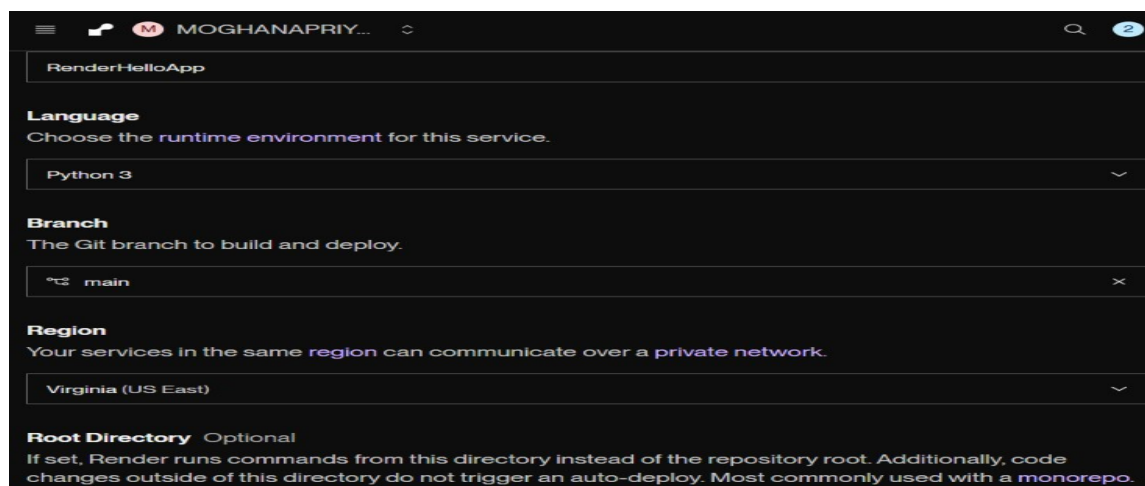
Step 2: Prepare the Application Repository

A sample web application was created and pushed to a new GitHub repository. The repository included the application source code along with the required start and build configuration (such as package.json for a Node.js application).

Step 3: Create a New Web Service on Render

From the Render dashboard, New → Web Service was selected, and the application's GitHub repository was connected and authorized for access.





RenderHelloApp

Language
Choose the runtime environment for this service.

Python 3

Branch
The Git branch to build and deploy.

main

Region
Your services in the same region can communicate over a private network.

Virginia (US East)

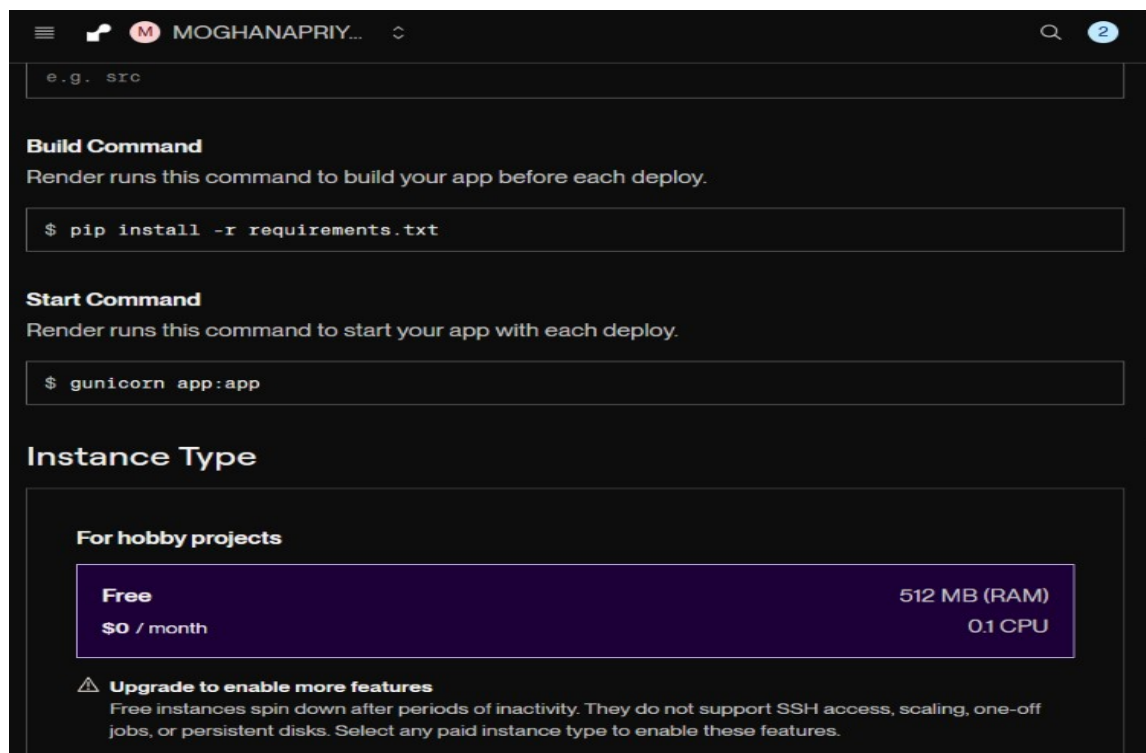
Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.

Step 4: Configure the Build and Start Settings

The build command and start command for the application were specified, the runtime environment was selected, and the free instance type was chosen for deployment.

Build Command: `npm install`

Start Command: `npm start`



e.g. src

Build Command
Render runs this command to build your app before each deploy.

\$ pip install -r requirements.txt

Start Command
Render runs this command to start your app with each deploy.

\$ gunicorn app:app

Instance Type

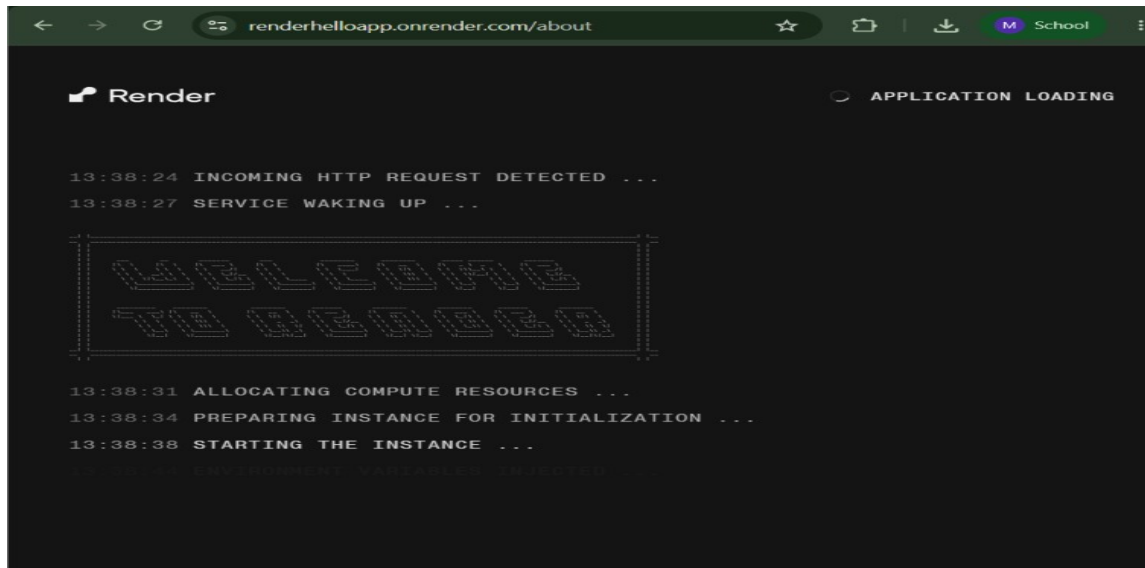
For hobby projects

Free \$0 / month	512 MB (RAM) 0.1 CPU
----------------------------	-------------------------

Upgrade to enable more features
Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

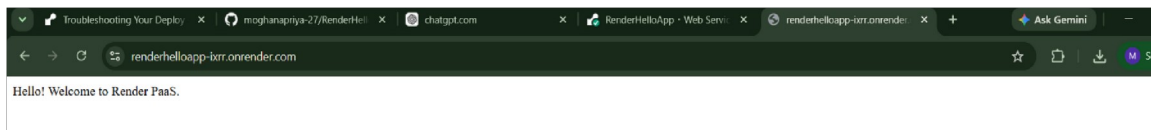
Step 5: Deploy the Web Service

The Create Web Service option was selected. Render automatically pulled the source code, installed the dependencies, built the application, and deployed it to its managed infrastructure.



Step 6: Access and Verify the Deployed Application

Once the deployment completed successfully, Render generated a public URL ending with onrender.com. The application was accessed through this URL in a web browser, and the deployment logs and service status were verified from the Render dashboard.



Conclusion

A web application was successfully deployed on the Render PaaS platform. The experiment demonstrated how a Platform-as-a-Service model automates build, deployment, and hosting directly from a source-code repository, without requiring manual server or infrastructure management, thereby meeting the objectives of the experiment.